

任务三：横向条形图完整解析

第一部分：代码逐段详解

1. 准备工作：修好“基建”和“材料”

```
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 魔法代码 1：让图表能显示中文
plt.rcParams['axes.unicode_minus'] = False # 魔法代码 2：让负号正常显示
```

数据准备

```
windows = ['面食窗口', '套餐窗口', '小吃窗口'] # 分类名称（未来会放在 Y 轴）
avg_times = [106.7, 163.3, 60.0] # 对应的排队时间（未来会放在 X 轴）
```

详解：作图前，先把画笔（plt）请出来，并设置好中文字体。接着，准备好两组数据，注意它们的顺序是一一对应的（面食窗口对应 106.7 秒）。

2. 开始作图：铺画布与画核心柱子

```
plt.figure(figsize=(8, 5)) # 创建一张 宽 8、高 5 的画布# 核心绘图！
plt.barh(windows, avg_times, color='lightcoral', edgecolor='black')
```

详解：

barh 中的 h 代表 horizontal（水平）。因为窗口名字太长，如果用普通的竖向柱状图（bar），文字会挤在一起看不清，所以改成横向拉伸。

注意参数顺序：plt.barh(Y 轴分类, X 轴数值)。color 设置了柱子内部颜色为浅珊瑚色，edgecolor 给柱子画了一圈黑边，显得更立体。

3. 图表精装修：加标题、打标签、拉长坐标轴

```
plt.title('食堂各窗口平均排队时间对比', fontsize=15)
plt.xlabel('排队时间（秒）', fontsize=12)
plt.ylabel('窗口名称', fontsize=12)
plt.xlim(0, 200) # 强制设置 X 轴的范围是从 0 到 200
```

详解：前三行是常规的贴标签。最后一行 plt.xlim(0, 200) 非常关键！因为最大的数据是 163.3，如果不设置，X 轴可能到 170 就结束了。强制拉长到 200，是为了给等会儿要写在柱子右边的数字留出“呼吸空间”。

4. 高阶操作：在柱子旁边写上具体的数字

```
codePython
for i in range(len(windows)):
    plt.text(avg_times[i] + 3, i, f'{avg_times[i]:.1f}', va='center', fontsize=10)
```

详解：这是全场最难的一句！for 循环会让下面的代码执行 3 次（对应 3 个窗口）。

参数 1 (X 坐标): avg_times[i] + 3。意思是数字放在柱子顶端再往右偏移 3 个单位。不加 3 的话，数字会紧紧贴着柱子边缘，很难看。

参数 2 (Y 坐标): i。在横向条形图中，第 0 个、第 1 个、第 2 个柱子的中心线 Y 坐标刚好

就是 0, 1, 2。

参数 3 (文字内容): `f'{avg_times[i]:.1f}'`。使用了 Python 的 f-string 格式化大法, `.1f` 意思是强制保留 1 位小数 (即使是 60, 也会显示成 60.0)。

参数 4 (对齐方式): `va='center'`。Vertical Alignment (垂直对齐), 确保写出来的数字和柱子的中心线在同一水平高度上。

写这段代码时, 请特别警惕以下 4 个“雷区”:

易错点 1: 把 `barh` 写成了 `bar`

错误做法: `plt.bar(windows, avg_times)`

后果: 图表变成了竖向的柱状图, 而且因为下面写标签的代码是按横向图的逻辑写的, 最后的数字标注会直接飞到天上去, 图表大乱。

口诀: 横向长条一定要带 `h`!

易错点 2: 打标签时, 搞反了 X 和 Y 坐标

错误做法: `plt.text(i, avg_times[i] + 3, ...)`

后果: 这是画竖向柱状图打标签的公式! 在横向图中套用这个公式, 数字会全跑到图表的左下角挤成一团。

记住: 横向图中, X 决定远近 (数值大小), Y 决定上下 (第几个柱子)。所以第一参数必须是长度, 第二参数必须是索引 `i`。

易错点 3: 忽略了 `plt.xlim(0, 200)`

错误做法: 觉得这行代码没用, 直接删掉。

后果: “套餐窗口”的柱子很长 (163.3), 你又要求在它右边加数字。如果图表右边没有留白, 最长那个柱子右边的数字就会被图表的右边缘直接切掉一半, 显示不全。

易错点 4: 字符串格式化符号写错

错误做法: 写成 `f'{avg_times[i]:.1f}'` (漏了小数点) 或者不写 `f` 直接变成 `{avg_times[i]:.1f}`。

后果: 直接报错, 或者图表上直接显示出一长串代码原文。

牢记: f-string 必须以 `f` 开头, 花括号 `{}` 里面装变量, 冒号 `:` 后面跟格式, `.1f` 的点 `.` 绝对不能少!

直方图完整解析 (数据分布)

第一部分: 代码逐段详解

1. 准备工作: 修好基建

`import matplotlib.pyplot as plt`
`import numpy as np` # 导入了 numpy 库 (虽然这段代码其实没用到它, 但带着也无妨)

`plt.rcParams['font.sans-serif'] = ['SimHei']` # 允许显示中文

`plt.rcParams['axes.unicode_minus'] = False` # 允许显示负号

详解: 这是每次画图的标准起手式。

2. 数据处理：把所有数据“倒进一个大盆里”

```
noodle_times = [120, 90, 110]
```

```
setmeal_times = [180, 150, 160]
```

```
snack_times = [60, 50, 70]# 核心操作：把三个列表无缝拼接成一个长列表
```

```
all_times = noodle_times + setmeal_times + snack_times
```

详解：画直方图时，我们不再关心“哪个时间属于哪个窗口”，而是关心“所有排队时间放在一起是什么形态”。在 Python 中，列表加上列表 (+)，等于把它们首尾相连拼成一个大列表。

合并后的 all_times 长这样：[120, 90, 110, 180, 150, 160, 60, 50, 70]（共 9 个数据）。

3. 核心绘图：让程序帮你“自动分拣”

```
plt.figure(figsize=(8, 5))# 核心神仙代码！
```

```
plt.hist(all_times, bins=5, color='purple', edgecolor='black', alpha=0.7)
```

详解：

plt.hist：绘制直方图的专属函数。它非常聪明，你只要把那 9 个乱糟糟的数字扔给它，它会自动帮你分类。

bins=5：直方图的灵魂参数！意思是：“请把最短到最长的时间，均匀切成 5 个区间（桶），然后数一数每个桶里掉进去了几个数字，画成 5 根柱子”。

alpha=0.7：设置颜色的透明度为 70%，让紫色看起来不那么刺眼。

edgecolor='black'：给每根柱子画上黑色边框。

4. 图表精装修：加标签与辅助线

```
plt.title('食堂排队时间分布直方图',fontsize=15)
```

```
plt.xlabel('排队时间区间（秒）',fontsize=12)
```

```
plt.ylabel('频数（测量次数）',fontsize=12)
```

```
# 添加网格线
```

```
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

```
plt.show()
```

详解：直方图的 Y 轴代表的是“频数”（即出现了几次）。为了方便肉眼看清某根柱子到底是 1 次还是 2 次，我们用 plt.grid 加了网格线。

axis='y'：只显示水平方向的横线（Y 轴网格），不要竖线，这样画面更干净。

linestyle='--'：把实线变成虚线。

这是整个第三章中，同学们报错率最高的一段代码，请千万避开以下雷区：

易错点 1：把“直方图”当成了“柱状图”来画（致命错误）

错误做法：新手总想给 X 轴和 Y 轴都塞数据，写成 plt.hist(windows, all_times)。

后果：程序瞬间崩溃报错！

牢记：柱状图（bar）需要明确的 X 和 Y；而直方图（hist）只需要传入一个装满数字的一维大列表即可！你只要给数据，统计和画高低柱子的工作，hist 函数在底层帮你做完了。

易错点 2：忘记写 edgecolor='black'

错误做法: `plt.hist(all_times, bins=5, color='purple')`

后果: 代码不会报错, 但画出来的图极丑。因为直方图的柱子是紧紧挨在一起的, 如果不加黑边框, 5 根紫色柱子会融合成一大坨紫色的长方形色块, 你根本看不清到底分了几个区间。

易错点 3: 错误理解列表的 + 号

内心戏: 以为 `[1, 2] + [3, 4]` 的结果是 `[4, 6]` (两个数字相加)。

真实情况: 在 Python 普通列表里, + 号代表拼接, 结果是 `[1, 2, 3, 4]`。千万不要以为这里是在做排队时间的数学加法!

易错点 4: bins 参数乱设一通

错误做法: 觉得数字越大越好, 设成 `bins=50`; 或者忘了写这个参数。

后果:

如果不写, 系统会默认分成 10 份, 对于只有 9 个数据的我们来说, 图表会显得很稀疏空洞。

如果设成 50 份, 图表会变成梳子一样, 全是细细的线, 完全失去了“看集中分布趋势”的意义。

牢记: `bins` (箱子数量) 要根据数据总量来凭感觉调整。数据少 (十几二十个), `bins` 设 5 到 8 即可; 数据多 (成百上千个), `bins` 可以设到 20 甚至更多。多改几次数值看看效果!